Veridia: manufacturer matching

What I built

I built a tool for a buyer or sourcing engineer who has a manufacturing requirement and needs a sensible starting list of suppliers. For the German buyer in the brief, that means finding companies with the right material, process, order size and certification, then checking whether the timing is plausible. The CSV can help with that first screen. It cannot tell the buyer who is ready to accept an order today.

The requirement form sits beside the results, with the two assignment examples available as presets. Each supplier has an explanation tied to the source data. I added comparison for up to three suppliers and a checklist of questions to take into a supplier conversation. Those additions help the buyer act on the shortlist: confirm the grade, drawing, tolerances, certificate documents and production slot.

How the shortlist is decided

I check eligibility before ranking. A supplier needs an unambiguous, active identity, the requested material and processes, a suitable order range, the required certificates and a known lead time within the limit. Among suppliers that pass, shorter production time comes first, followed by delivery history, export experience and manufacturer ID. The order is visible and repeatable; there is no weighted score hiding why one supplier ranks above another.

Timing needed a separate decision. The request stores an evaluation date, and certificates must remain valid through estimated production completion, assuming work starts on that date. The expiry day counts as valid. The file gives one expiry date per supplier, so I apply it to the listed certificates and ask the buyer to verify the documents. Later repeat orders can raise a renewal warning even when the first batch qualifies.

Production completion and arrival in Germany are separate. The dataset has no transit or customs times. When the buyer asks for arrival within six weeks, the tool can screen production timing but leaves arrival unconfirmed. It does not turn a production lead time into a delivery promise.

Each browser tab has its own buyer state and matching worker. Editing a requirement marks the old results as outdated and clears comparison and checklist progress; a late response from an earlier request cannot replace newer work. Saved JSON keeps the inputs, evidence, questions, progress and dataset/policy versions. Reopening it shows the saved decision. Recalculating is explicit, and progress survives only when the evidence is unchanged.

The choice I would defend

Ten manufacturer IDs each refer to two different companies in the supplied CSV. I kept all twenty affected rows out of the shortlist. Picking whichever row came first would be easy, but then the order of the CSV would decide which company the buyer sees. Merging the rows could be worse: it could create a supplier with capabilities drawn from two companies.

That leaves 20 of the 1,000 rows unusable until their identities are resolved. I think that is preferable to recommending a company whose identity is uncertain. The original rows and their locations in the CSV remain available for correction. Missing lead times and impossible delivery percentages also stay unknown; they do not become zero or favourable ranking values.

Where the tool stops

With the evaluation date fixed at 27 October 2026, Buyer A gets 33 preliminary matches and Buyer B gets only VM1006. Both buyers can see VM1006. Their combined batches total 2,500 units, against its listed monthly capacity of 1,500. That does not settle whether both orders fit into six weeks: current bookings, start dates and available production slots are missing. The next step is to confirm a schedule with the supplier. The application does not reserve capacity.

I left out login, supplier messaging, live bookings, quotes and freight estimates. They need data or workflows beyond this snapshot. A static React/TypeScript app with local matching is enough for the assignment and has no backend or API cost. I also kept language models out of the matching path: the supplied fields support explicit rules, and an AI-generated explanation would not verify missing supplier facts.

How I would know it is working

The first check is correctness: every shortlisted supplier should pass the stated rules. The interface should never mix two buyers' results or present an older request's result as current. The project has 47 passing unit tests and a successful 19-test Chromium run covering matching, dates, report handling, independent sessions and frontend interactions. An independent comparison against the original CSV checked 244 requests without a shortlist or ranking mismatch. These are checks against the mock dataset, not evidence that suppliers can fulfil real orders.

In a pilot, I would measure how long it takes a buyer to reach a useful shortlist, how often a sourcing expert accepts it, and how many shortlisted suppliers confirm the requirement. I would also review no-match cases to see whether the rules or the data need attention. Returning more names would not be enough; the shortlist needs to make the next supplier conversation easier.